

PROGETTAZIONE LOW POWER

27006347 (A.A. 2024-25)

Ottimizzazione della Potenza Dinamica di un Circuito RTL Sintetizzato

Replica, verifica e analisi comparativa di cinque architetture
low-power su FPGA

Progetto 03

Stefano Scarcelli — M. 269231

10/07/2026

Indice

1	Obiettivo e Specifiche del Progetto	3
1.1	Vincoli di progetto	3
1.2	Struttura dell'elaborato	3
2	Circuito di Riferimento e Metodologia	4
2.1	Introduzione	4
2.2	Il Circuito di Riferimento	4
2.3	Ambiente di Verifica: Testbench Auto-Validante	5
2.4	Flusso di Automazione: Tcl e Batch	6
3	Tecniche di Ottimizzazione Implementate	8
3.1	Introduzione	8
3.2	Step 01 — Baseline	8
3.3	Step 02 — Pipeline Registering	9
3.4	Step 03 — Pre-Computing Reordering	10
3.5	Step 04 — Pre-Computing Pipeline (Reordering & Registering)	10
3.6	Step 05 — Gated Pre-Computing (Isolated Reordering)	11
4	Confronto Cross-Configurazione	13
4.1	Step 06 — Analisi Comparativa	13
4.1.1	Metodologia	13
4.1.2	Risorse Hardware (LUT / FF)	13
4.1.3	Temporizzazione (fmax)	14
4.1.4	Ripartizione della Potenza	14
4.1.5	Confronto Relativo (@ 10 ns)	15
4.1.6	Trade-off Architettureali	16
5	Conclusioni	17

Capitolo 1 — Obiettivo e Specifiche del Progetto

L'obiettivo del progetto è l'applicazione di tecniche di ottimizzazione low-power applicate a un circuito di riferimento con sommatore *RTL sintetizzati*. La scelta delle tecniche usate si rifanno a quelle viste a lezione come **Reordering**, **Registering**, **Gated Inputs**, **Precomputing** e **Clock Gating**. Questo progetto introduce un flusso **RTL/FPGA** completo: descrizione in VHDL, sintesi e implementazione in «Vivado», verifica funzionale tramite testbench auto-validante, e caratterizzazione di area, temporizzazione e potenza dinamica su un intero sweep di frequenze di clock.

1.1 Vincoli di progetto

Il progetto rispetta i seguenti vincoli:

1. **Topologia di riferimento:** un sommatore a 32 bit che seleziona tra le coppie di operandi (A, C) o (B, D) tramite un percorso di controllo basato sulla parità della somma $sel_1 + sel_2$ (a 8 bit), elaborata da un Ripple Carry Adder (RCA8) e un albero XOR (`parity_check`).
2. **Toolchain:** descrizione RTL in VHDL (IEEE `std_logic_1164/numeric_std/math_real`), sintesi e implementazione in Vivado, orchestrazione tramite script Tcl e un batch Windows (`run_sims.bat`) per l'esecuzione automatizzata dell'intera matrice `varianti × clock-constraint`.
3. **Equivalenza funzionale:** ogni variante architetturale deve preservare bit-esattamente il comportamento del circuito di riferimento (selezione e somma corrette), verificata da un **testbench comune e auto-validante**, indipendente dalla profondità di pipeline di ciascuna variante.
4. **Clock-constraint sweep:** area, temporizzazione e potenza dinamica sono caratterizzate su uno **sweep di 5 periodi di clock** (10, 9, 8, 7, 6 ns), non su un singolo vincolo statico.
5. **Potenza dinamica:** stimata in post-implementazione tramite attività di commutazione reale (file `.saif`, generato dalla simulazione `timing post-implementation`).

1.2 Struttura dell'elaborato

Il lavoro è organizzato in **tre fasi**, corrispondenti agli step implementati nel notebook Python:

- **Circuito di riferimento e metodologia** (Capitolo 2): descrizione del circuito base, dell'infrastruttura di verifica e del flusso di automazione Tcl/batch.
- **Tecniche di ottimizzazione** (Capitolo 3, Step 01–05): implementazione e caratterizzazione individuale delle 5 varianti architetturelle identificate.
- **Confronto cross-configurazione** (Capitolo 4, Step 06): analisi comparativa di area, temporizzazione e potenza su tutte le varianti e l'intero sweep di clock, con validazione rispetto alla metodologia di riferimento.

Capitolo 2 — Circuito di Riferimento e Metodologia

2.1 Introduzione

Tutte le varianti architetturali condividono lo stesso insieme di componenti RTL riutilizzabili e la stessa interfaccia di porte (`clk`, `rst`, `a`, `b`, `c`, `d` a 32 bit, `sel1`, `sel2` a 8 bit e `z` a 33 bit) — condizione necessaria per poter verificare tutte le varianti con un **unico testbench**, e per rendere il confronto tra le architetture significativo a parità di interfaccia esterna.

2.2 Il Circuito di Riferimento

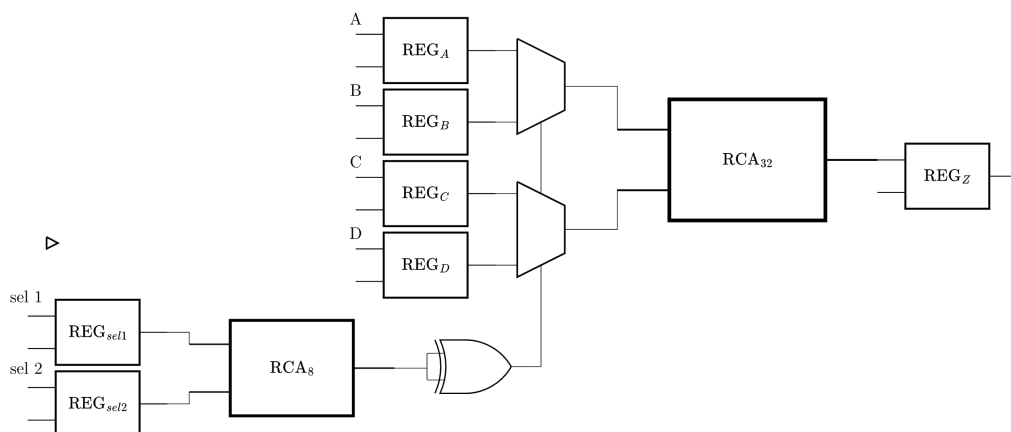
Il circuito **Baseline** (`top_baseline.vhd`) implementa l'architettura di base del circuito: un primo stadio di registri campiona operandi e segnali di selezione; il percorso di controllo (`AdderSel` → `Pattern`) deriva combinatoriamente il segnale `Z`; due multiplexer (`MUX_AB`, `MUX_CD`) selezionano la coppia di operandi **prima** della somma, eseguita da un unico Ripple Carry Adder a 32 bit (`Adder32EF`); il risultato è campionato nel registro di uscita `Z_reg`.

I blocchi condivisi (`res/srcs/project-03/vivado_libs/`, invariati rispetto alla loro prima implementazione) sono:

- `full_adder.vhd`: full-adder a 1 bit, combinatorio.
- `rca_gen.vhd`: Ripple Carry Adder generico, costruito come catena strutturale di `full_adder` tramite `generate`:

```
adder_chain : for i in 0 to WIDTH - 1 generate
begin
    stage_i : full_adder
        port map (
            a    => a(i),
            b    => b(i),
            cin  => carry_chain(i),
            sum  => sum(i),
            cout => carry_chain(i + 1)
        );
end generate adder_chain;
```

- `parity_check.vhd`: albero XOR generico, usato sia per il percorso di controllo (`Pattern`) sia come modello di riferimento comportamentale nel testbench.
- `mux2.vhd`: multiplexer 2-to-1 generico, riusato per `MUX_AB`/`MUX_CD` e, nelle varianti `Reordering`, per `MUX_SUM`.
- `rtl_reg_sync.vhd`: registro generico con **reset asincrono attivo-alto** e un ingresso opzionale `ce` (clock enable, default '1'), retrocompatibile con tutte le istanze preesistenti — necessario per il **Clock Gating** della variante `Isolated Reordering`.
- `operand_gate.vhd`: blocco di isolamento operandi (**Gated Input**), forza l'uscita a zero quando il segnale di `gate` è deassertito.

Figura 1: Schematic RTL elaborato del circuito **Baseline**.

2.3 Ambiente di Verifica: Testbench Auto-Validante

Le cinque varianti hanno profondità di pipeline diverse (Baseline/Reordering: 2 cicli; Registering/Reordering & Registering/Isolated Reordering: 4 cicli). Per evitare di duplicare il testbench una volta per variante, `tb_top.vhd` implementa un unico corpo **agnostico rispetto alla latenza**, parametrizzato tramite due generic:

- **PIPELINE_LATENCY**: profondità della pipeline di riferimento auto-validante (shift register dei valori attesi), impostata per variante dallo script Tcl.
- **VARIANT_ID**: seleziona quale top-level istanziare come DUT (*Device Under Test*), tramite blocchi `if-generate`:

```
gen_base: if VARIANT_ID = 0 generate
    dut: top_baseline port map (clk=>clk, rst=>rst, a=>a, b=>b, c=>c, d=>d,
                               sel1=>sel1, sel2=>sel2, z=>z);
end generate;

gen_reg: if VARIANT_ID = 1 generate
    dut: top_registering port map (clk=>clk, rst=>rst, a=>a, b=>b, c=>c, d=>d,
                                   sel1=>sel1, sel2=>sel2, z=>z);
end generate;

-- gen_reord (VARIANT_ID = 2), gen_reord_reg (VARIANT_ID = 3),
-- gen_isol (VARIANT_ID = 4): stessa struttura, non riportati per brevità.
```

Il modello di riferimento ($Z = A+C$ quando `parity(sel1+sel2)` è dispari, altrimenti $B+D$) è calcolato in VHDL con la stessa logica XOR-tree di `parity_check.vhd`, e confrontato con l'uscita del DUT tramite una pipeline di valori attesi a profondità `PIPELINE_LATENCY`, che si auto-allinea alla latenza specifica di ciascuna variante senza richiedere logica di sincronizzazione aggiuntiva:

```
check : process (clk) is
begin
    if rising_edge(clk) then
        if valid_pipeline(PIPELINE_LATENCY) = '1' then
            check_count <= check_count + 1;
            if z /= expected_pipeline(PIPELINE_LATENCY) then
                error_count <= error_count + 1;
                report "ERROR: [TB] MISMATCH at check " & integer'image(check_count)
                    severity error;
            end if;
        end if;
    end if;
end process;
```

```

        end if;
    end if;
end process check;

```

Al termine (`std.env.stop`, VHDL-2008), il testbench riporta un riepilogo [TB] *N checks*, *M errors* seguito da [TB] *PASS*/[TB] *FAIL*, testo cercato dal notebook Python nei log di simulazione per la verifica automatica dello Step corrispondente.

2.4 Flusso di Automazione: Tcl e Batch

Ogni combinazione (variante, periodo di clock) è eseguita da `scripts/project-03/run_variant.tcl`, invocato in modalità batch. Due aspetti del flusso, emersi durante lo sviluppo, meritano di essere evidenziati:

Vincolo di clock. I file `.xdc` supportano solo un sottoinsieme ristretto di Tcl (`set`, `list`, `expr`, più i comandi di constraint) — non `if`, non `info exists` — e `launch_runs` esegue sintesi/implementazione in un contesto che **non** condivide le variabili Tcl globali dello script chiamante. Per questo motivo, il periodo di clock non è parametrizzato tramite una variabile letta dal file `.xdc`, ma riscritto **letteralmente** su disco prima di ogni sintesi:

```

set xdc_fh [open $xdc_path w]
puts $xdc_fh "create_clock -period $clk_period_ns -name clk \[get_ports clk\]"
close $xdc_fh

```

Attività di commutazione (SAIF). La generazione manuale del file `.saif` (tramite `open_saif/log_saif/close_saif`) si è rivelata inaffidabile (0 net corrisposti su un primo tentativo). Il flusso adottato imposta invece le proprietà del fileset di simulazione prima di `launch_simulation`, delegando a `xsim` la generazione automatica di un SAIF con copertura completa dei segnali:

```

set_property -name {xsim.simulate.saif} -value $saif_filename -objects [get_filesets sim_1]
set_property -name {xsim.simulate.saif_all_signals} -value {true} -objects [get_filesets
sim_1]
set_property -name {xsim.simulate.runtime} -value {all} -objects [get_filesets sim_1]

launch_simulation -mode post-implementation -type timing -simset [get_filesets sim_1]

...

read_saif $saif_path
report_power -file [file join $reports_dir "power.rpt"] -xpe [file join $reports_dir
"power.xpe"]

```

L'esportazione aggiuntiva in formato `.xpe` (XML, per Xilinx Power Estimator) permette al notebook di estrarre i contributi di potenza tramite parsing strutturato (`xml.etree.ElementTree`) anziché tramite regular expression sul report testuale, più robusto e con maggiore precisione dei dati letti.

I report (`utilization.rpt`, `timing_summary.rpt`, `power.rpt`, `power.xpe`, `activity.saif`) sono organizzati per combinazione variante-clock (`notebooks/output/project-03/sims/reports/<variante>_<clock>`), mentre i log di simulazione (usati per la verifica funzionale) risiedono in `notebooks/output/project-03/sims/logs/`. L'intera matrice di 5 varianti $\times$ 5 periodi di clock è eseguita da `run_sims.bat`:

```
for %%V in (%VARIANTS%) do (  
  for %%C in (%CLOCKS%) do (  
    set "LOG_FILE=%LOGS_DIR%\%%V_%%Cns.log"  
    call vivado -mode batch -log "!LOG_FILE!" -source "%TCL_SCRIPT%" -tclargs %%V %%C  
  )  
)
```

Capitolo 3 — Tecniche di Ottimizzazione Implementate

3.1 Introduzione

Questo capitolo presenta le 5 varianti architetturali, ciascuna verificata funzionalmente tramite il testbench comune (Capitolo 2) e caratterizzata sull'intero sweep di clock 10–6 ns.

Per ciascuna variante sono riportati: il principio di funzionamento, l'esito della verifica funzionale, e le metriche di area (LUT, FF), temporizzazione (WNS, fmax) e potenza dinamica estratte dai report **Vivado**.

3.2 Step 01 — Baseline

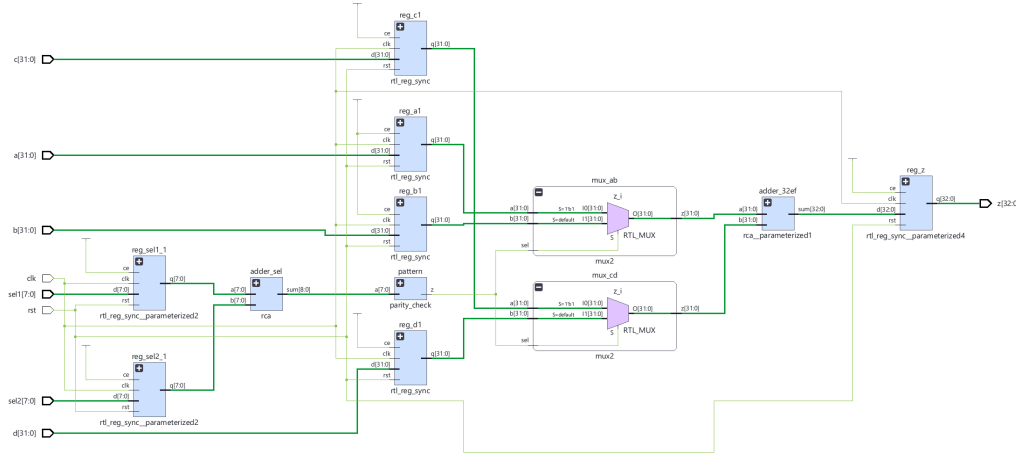
Circuito di riferimento (Capitolo 2): selezione degli operandi **prima** della somma, segnale di selezione combinatorio e “*non-latched*” — condizione che produce **glitch** sul multiplexer di selezione.

La verifica funzionale ha esito positivo (200/200 check) per tutti i periodi di clock tranne il più aggressivo, dove l'implementazione non converge:

Clock [ns]	LUT	FF	WNS [ns]	fmax [MHz]	Internal Power ¹ [mW]	Verifica
10	126	177	1.674	120.106	16.494	PASS
9	126	177	0.980	124.688	15.153	PASS
8	127	177	0.749	137.912	16.436	PASS
7	136	177	0.392	151.332	18.356	PASS
6	—	—	—	—	—	<i>Timing FAIL</i>

A 6 ns l'implementazione fallisce: il vincolo è troppo aggressivo per la topologia Baseline non ottimizzata. Questo esito, di per sé, è un primo indizio a favore delle tecniche successive: la sola **robustezza temporale** (non solo la potenza) beneficia dell'eliminazione del percorso combinatorio lungo prima della somma.

¹Logic + Signals + Clock, esclusi I/O e potenza statica.

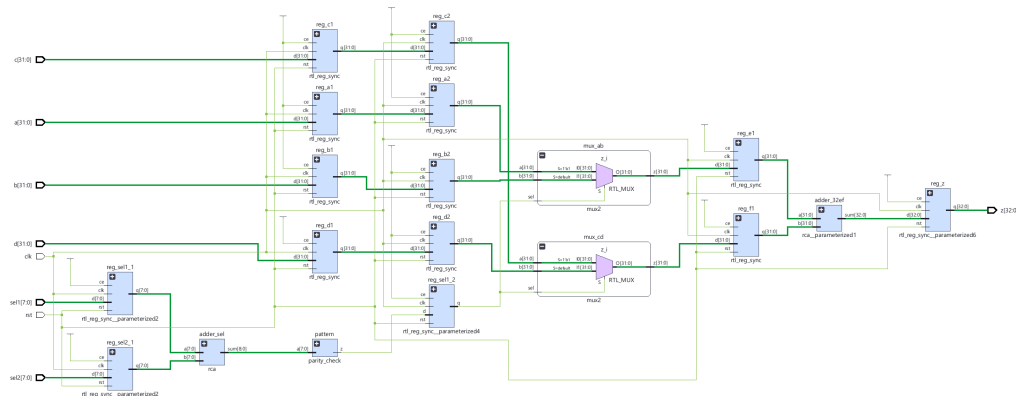
Figura 2: Schematic RTL elaborato per **Baseline** (Vivado).

3.3 Step 02 — Pipeline Registering

Aggiunge un secondo stadio di registri per operandi e segnale di selezione, e un ulteriore stadio che registra le **uscite** dei multiplexer (**E1_reg/F1_reg**) prima della somma. Il segnale di selezione registrato (**sel1_2_reg**) raggiunge il multiplexer stabile e privo di **glitch**.

Clock [ns]	LUT	FF	WNS [ns]	fmax [MHz]	Internal Power [mW]	Verifica
10	91	370	2.991	142.674	13.046	PASS
9	91	370	2.202	147.102	14.486	PASS
8	91	370	1.390	151.286	15.092	PASS
7	92	370	0.667	157.903	16.634	PASS
6	95	370	0.789	191.902	17.327	PASS

A parità di clock standard (10 ns), la potenza interna scende da 16.494 a 13.046 mW (−20.9%) e l'**f**_{max} sale da 120.1 a 142.7 MHz (+18.8%), a fronte di un incremento di **FF** (da 177 a 370, per i due stadi di pipeline aggiuntivi) e di una **riduzione** di LUT (da 126 a 91) — probabile conseguenza della minore pressione sul routing/logica combinatoria del percorso ora suddiviso in stadi più corti.

Figura 3: Schematic RTL elaborato per **Pipeline Registering** (Vivado).

3.4 Step 03 — Pre-Computing Reordering

Sostituisce il sommatore condiviso con due sommatori dedicati (**Adder32AC**, **Adder32BD**), calcolati parallelamente ogni ciclo; il risultato corretto è selezionato **dopo** la somma da **MUX_SUM**, pilotato dal segnale di selezione — ancora combinatorio e “*non-latched*” in questa variante.

Clock [ns]	LUT	FF	WNS [ns]	fmax [MHz]	Internal Power [mW]	Verifica
10	118	177	2.239	128.849	12.453	PASS
9	119	177	1.907	140.984	12.920	PASS
8	119	177	1.114	145.222	14.250	PASS
7	123	177	0.616	156.642	14.822	PASS
6	132	177	0.612	185.598	14.702	PASS

A 10 ns, il Reordering ottiene la potenza interna **più bassa fra tutte le varianti misurate** (12.453 mW, -24.5% rispetto al Baseline), con perfino un decremento delle LUT (da 126 a 118²) e nessun costo di FF aggiuntivo (177, invariato). A differenza del Baseline, l’implementazione converge anche a 6 ns.

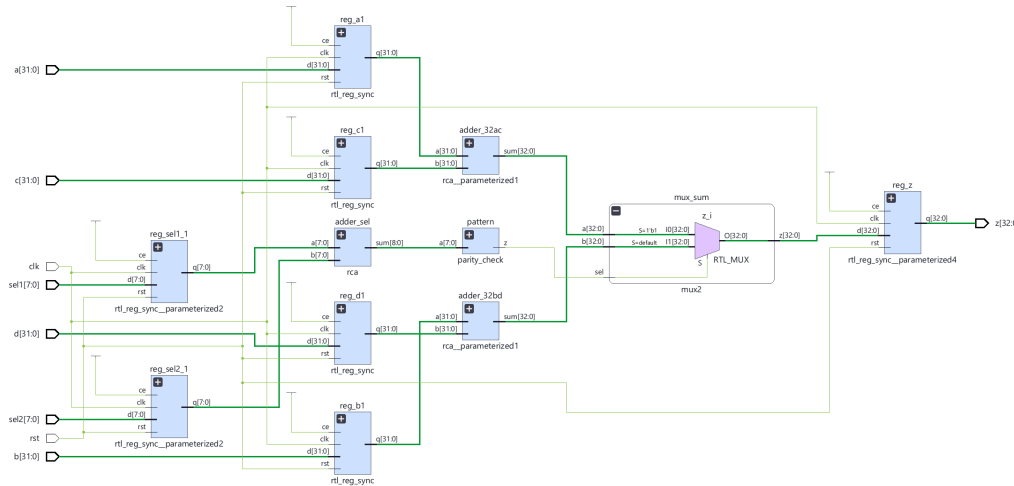


Figura 4: Schematic RTL elaborato per Pre-Computing Reordering (Vivado).

3.5 Step 04 — Pre-Computing Pipeline (Reordering & Registering)

Combina le due tecniche: gli operandi che alimentano **Adder32AC/Adder32BD** provengono da un secondo stadio di registri; le uscite dei due sommatori sono “*latched*” (**E1_reg/F1_reg**) prima di **MUX_SUM**; il segnale di selezione è riallineato tramite **due** registri in cascata (**sel1_2_reg** → **sel1_3_reg**), per restare sincronizzato con lo stadio di pipeline aggiuntivo introdotto dopo i sommatori.

²Il numero di LUT diminuisce leggermente rispetto al Baseline nonostante il sommatore aggiuntivo: la rimozione del multiplexer sugli operandi (32 bit, prima della somma) più che compensa il costo del secondo **Adder32**, poiché i multiplexer sugli operandi selezionavano due bus da 32 bit contro l’unico bus selezionato da **MUX_SUM** dopo la somma.

9	192	373	2.573	155.594	15.590	PASS
8	192	373	1.241	147.951	17.141	PASS
7	192	373	0.673	158.053	17.896	PASS
6	198	373	0.447	180.083	19.872	PASS

Il risultato è discusso in dettaglio nel Capitolo 4: l'ipotesi progettuale — ottenere la robustezza temporale del Reordering & Registering a un costo di potenza più vicino alla sola Registering — **non** è confermata dalle misure: il conteggio di LUT (192, contro i 128 della sola Reordering & Registering) e la potenza interna a 10 ns (15.169 mW, superiore sia alla sola Reordering che alla combinazione Reordering & Registering) indicano che il costo della logica di gating (`operand_gate` più il fan-out dei segnali di `ce`) supera, in questa implementazione, il risparmio di switching atteso.

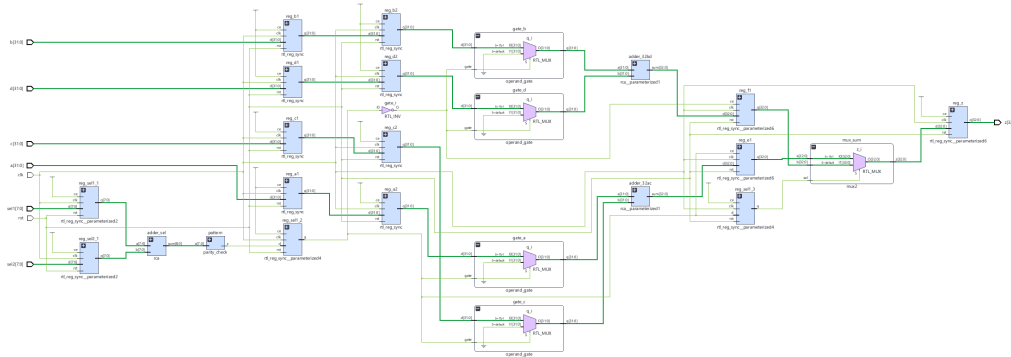


Figura 6: Schematic RTL elaborato per **Gated Pre-Computing** (Vivado).

Capitolo 4 — Confronto Cross-Configurazione

4.1 Step 06 — Analisi Comparativa

4.1.1 Metodologia

Il confronto aggrega le misure dei 5 Step precedenti sull'intero sweep di clock.

4.1.2 Risorse Hardware (*LUT* / *FF*)

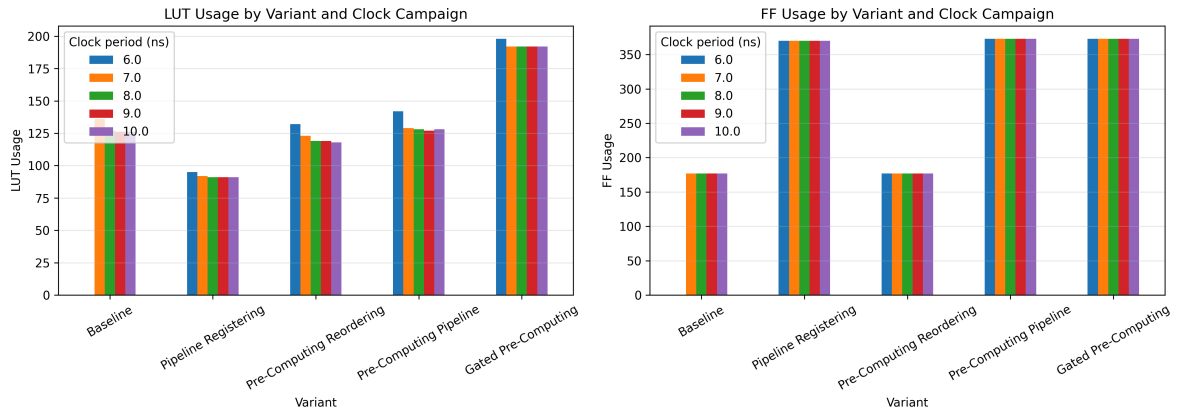


Figura 7: Utilizzo di LUT (sinistra) e FF (destra) per ciascuna variante, sull'intero sweep di clock.

Il dato più rilevante è l'incremento di LUT della variante Isolated Reordering (192–198), quasi il doppio rispetto a Reordering & Registering (127–142) a parità di FF (373 in entrambe): l'onere architetturale del gating ricade quasi interamente sulla logica combinatoria, non sui registri.

4.1.3 Temporizzazione (f_{max})

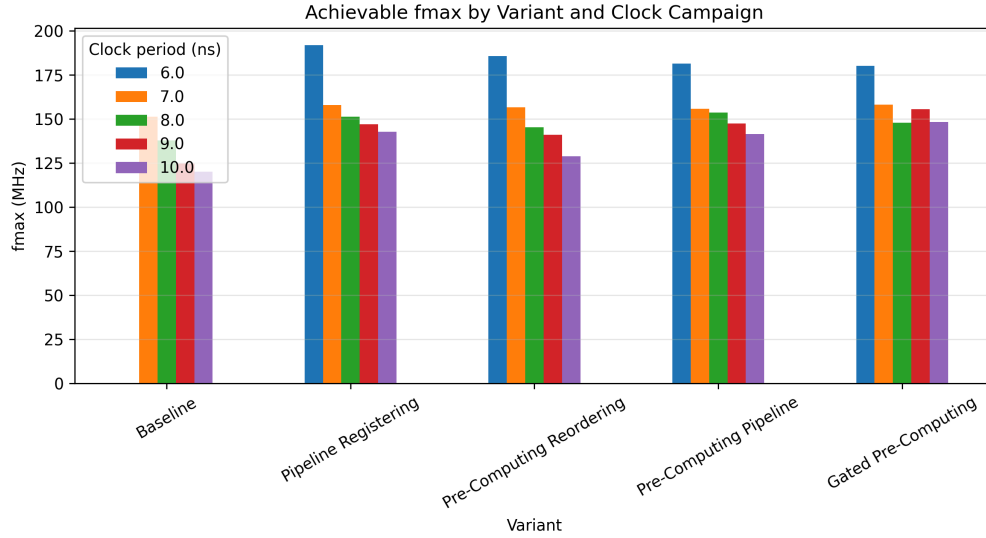


Figura 8: Frequenza massima raggiungibile per ciascuna variante, sull'intero sweep di clock.

Tutte e quattro le varianti ottimizzate chiudono la temporizzazione anche al vincolo più aggressivo (6 ns), a differenza del Baseline (che fallisce). La variante **Isolated Reordering** raggiunge il margine di temporizzazione (WNS) più ampio a 10 ns (3.260 ns, contro 2.991 della sola Registering e 2.928 della Reordering & Registering) — l'unico aspetto in cui l'investimento in logica di gating produce effettivamente il beneficio atteso.

4.1.4 Ripartizione della Potenza

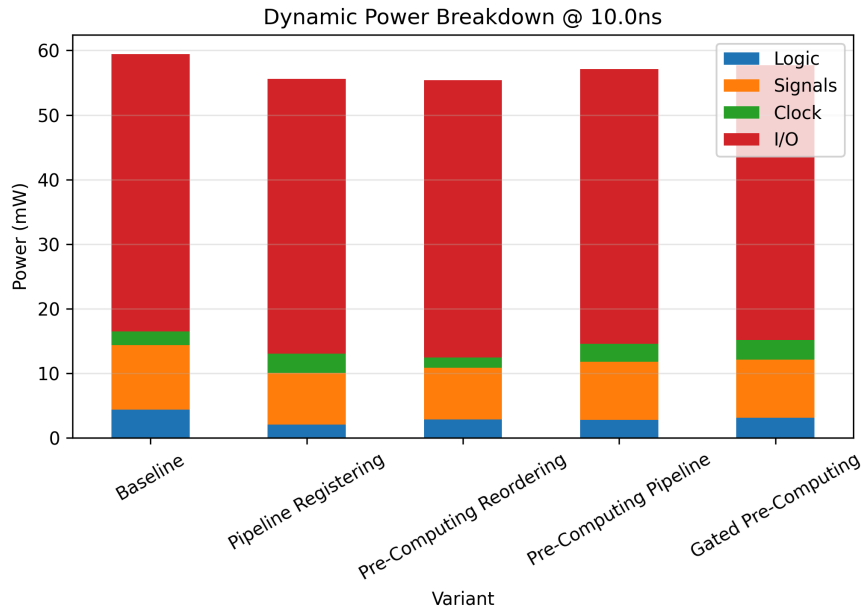


Figura 9: Ripartizione Logic/Signals/Clock/I-O a 10 ns.

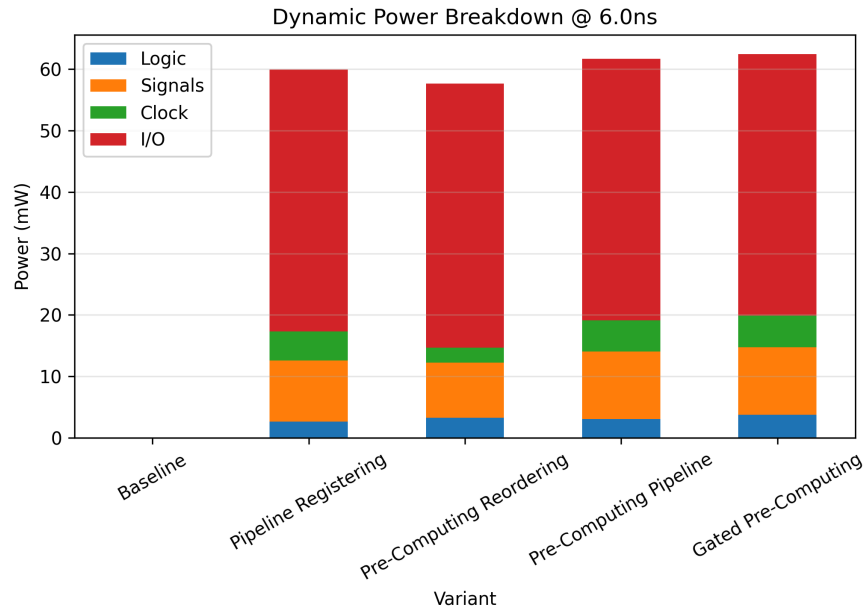


Figura 10: Ripartizione Logic/Signals/Clock/I-O a 6 ns.

Al diminuire del periodo di clock, il contributo **Clock** cresce in tutte le varianti registrate (es. Registering: da 3.000 mW a 10 ns a 4.704 mW a 6 ns), coerentemente con l'aumento della frequenza di commutazione della rete di distribuzione del clock — mentre il contributo **Logic** cresce più moderatamente, segno che il vincolo di temporizzazione più stringente forza il tool a percorsi combinatori diversi ma non necessariamente più commutanti.

4.1.5 Confronto Relativo (@ 10 ns)

Variante	Internal Power [mW]	Δ Power vs Baseline	fmax [MHz]	Δ fmax vs Baseline
Baseline	16.494	—	120.106	—
Pipeline Registering	13.046	−20.9%	142.674	+18.8%
Pre-Computing Reordering	12.453	−24.5%	128.849	+7.3%
Pre-Computing Pipeline	14.544	−11.8%	141.403	+17.7%
Gated Pre-Computing	15.169	−8.0%	148.368	+23.5%

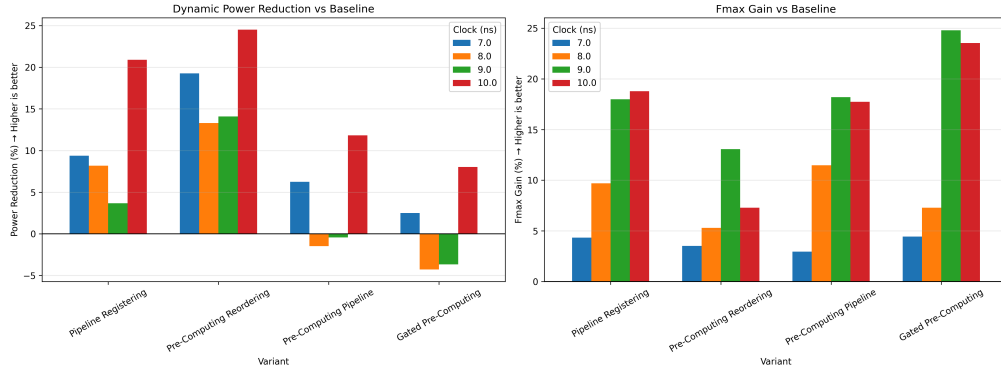


Figura 11: Riduzione di potenza dinamica e guadagno di fmax rispetto al Baseline (Dynamic Power esclude la potenza degli I/O).

L'ordinamento per riduzione di potenza (Reordering > Registering > Reordering & Registering > Isolated Reordering) è **l'opposto** di quanto ci si aspetterebbe da un ragionamento puramente additivo delle tecniche: combinarle, e poi estenderle con il gating, non produce un beneficio di potenza monotonicamente crescente. Il guadagno di fmax, al contrario, è massimo proprio per la variante più costosa in area (Isolated Reordering), a conferma che le due tecniche aggiuntive (Gated Inputs, Clock Gating) in questa implementazione ottimizzano prevalentemente il **marginale di temporizzazione**, non la potenza.

4.1.6 Trade-off Architeturali

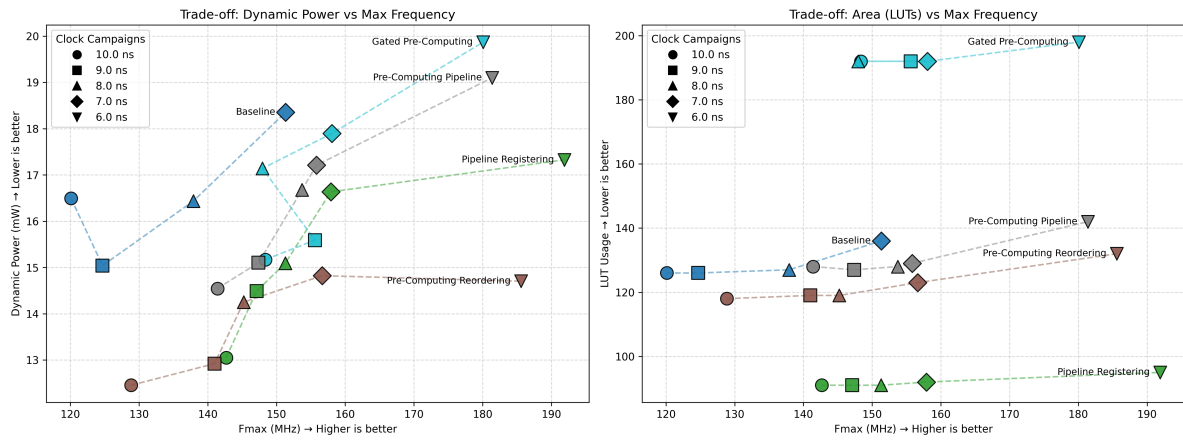


Figura 12: Sinistra: potenza dinamica vs fmax; Destra: LUT vs fmax.

Il grafico di trade-off evidenzia come Reordering domini Pareto-efficacemente su Reordering & Registering e Isolated Reordering nella coppia (potenza, fmax) fino al punto in cui la sola Registering non viene comunque superata in fmax: nessuna variante ottimizzata domina simultaneamente tutte le altre su area, potenza e temporizzazione — la scelta finale dipende dall'obiettivo di progetto (minima potenza: Reordering; massimo margine temporale: Isolated Reordering; minima area: Registering).

Capitolo 5 — Conclusioni

La replica del circuito e delle tecniche di Registering e Reordering conferma qualitativamente i risultati teorici: entrambe le tecniche riducono la potenza interna rispetto al Baseline (-20.9% e -24.5% rispettivamente, a 10 ns) e migliorano la temporizzazione, con il Baseline che non riesce nemmeno a chiudere l'implementazione al vincolo di clock più aggressivo testato (6 ns) mentre tutte le varianti ottimizzate vi riescono.

La combinazione delle due tecniche (Reordering & Registering) e, soprattutto, l'estensione originale con Gated Inputs, Precomputing e Clock Gating (Isolated Reordering) producono invece un risultato meno intuitivo: l'aggiunta di logica di isolamento e clock gating, pur teoricamente motivata dalla possibilità di precomputare il segnale di selezione, si traduce in un incremento di area (LUT quasi raddoppiati rispetto alla sola Reordering & Registering) **senza** un corrispondente beneficio di potenza — anzi, la potenza interna di Isolated Reordering risulta la più alta fra le quattro varianti ottimizzate. L'unico beneficio netto e misurabile dell'estensione è un margine di temporizzazione (WNS) più ampio, non la riduzione di potenza che ne aveva motivato la progettazione.

Questo risultato, seppur negativo rispetto all'ipotesi di partenza, è di per sé un'indicazione metodologicamente utile: l'overhead della logica di controllo introdotta per abilitare una tecnica low-power (qui, i segnali di gate e clock-enable e il relativo fan-out) può superare il risparmio di switching che la tecnica si propone di ottenere, specialmente su circuiti di dimensioni contenute come quello analizzato — un effetto che riportate a un solo punto di clock, non permettono di isolare con la stessa chiarezza offerta dallo sweep completo su cinque periodi di clock. Un possibile sviluppo futuro è la caratterizzazione dello stesso circuito su un dimensionamento maggiore (es. operandi a 64 o 128 bit), dove il costo relativo della logica di controllo rispetto al datapath si riduce, per verificare se il vantaggio di Isolated Reordering diventi effettivamente misurabile in potenza oltre che in temporizzazione.